TP05 Report

Real-Time Data Streaming and Processing Using Apache Kafka

University of El Oued
Faculty of Exact Sciences — Department of Computer Science
Master 2: IA & Data Science | Module: Big Data Analytics | 2026/2027

Student: Alya Toumi — G1

---

1. Objective and setup

This practical work demonstrates event streaming with Apache Kafka in a simulated smart-campus sensor-monitoring system. Python producers publish JSON temperature and humidity events, and Python consumers read the stream, detect high temperatures, calculate statistics, filter by building, and export events to CSV.

The environment used Windows, VS Code, Python 3.11, `kafka-python==2.0.2`, Docker Compose, and `apache/kafka:3.9.1`. Kafka ran in KRaft mode as one broker (`tp05-kafka`), exposed at `localhost:9092`. The original `producer.py` and `consumer.py` were preserved; investigation scripts were created separately.

2. Experiments and observed results

Basic producer and consumer

The original producer sent 30 simulated sensor events to `campus-sensors`. The consumer decoded JSON and displayed sensor ID, timestamp, temperature and humidity. Readings at or above **35 °C** triggered an alert. The topic was configured with **3 partitions** and **replication factor 1**.

Experiment 1 — 100 events

A separate `producer_100.py` published 100 events. Together with the original 30 events, the topic contained 130 events for the subsequent historical reading test. This showed that the same pipeline can handle a larger event batch without changing the broker configuration.

Experiment 2 — Running average

`consumer_average.py` calculated the cumulative mean temperature as events arrived:

$$running_average = \frac{total_temperature}{event_count}.$$

It processed **130 events**, producing a final average of **29.88 °C** and **40 high-temperature alerts**. This is an incremental stream calculation, not a fixed-window average.

Experiment 3 — Consumer groups

Consumers using `group-A` and `group-B` independently read the same 130 stored events. Two consumers sharing `group-shared` instead shared partition assignments: one was assigned **Partition 2**, and the other **Partitions 0 and 1**, verified using `kafka-consumer-groups.sh -describe -members -verbose`. The test confirmed partition assignment, rather than measuring throughput.

Experiment 4 — One versus three partitions

Because Kafka cannot directly decrease a topic's partition count, separate topics were created: `tp05-one-partition` (1 partition) and `tp05-three-partitions` (3 partitions), both with replication factor 1. Twelve events were sent to each.

Topic / partition	Events	Observed offsets
One-partition topic: P0	12	0–11
Three-partition topic: P0	2	0–1
Three-partition topic: P1	7	0–6
Three-partition topic: P2	3	0–2

Table 1: Event distribution for the one- and three-partition topics.

The three-partition distribution was uneven, which is permissible. Offsets are independent per partition, and ordering is guaranteed within a partition, not globally across a multi-partition topic. All partitions remained on **broker 1**, so adding partitions did not add broker-level redundancy.

Experiment 5 — Building filtering

`producer_buildings.py` published 30 events to `campus-buildings`, adding the JSON field `campus_building`. There were 10 events each for Building-A, Building-B and Building-C. `consumer_buildings.py` received all **30** but displayed only the **10** belonging to Building-A. Among the matching events, **3** temperatures met the 35 °C alert threshold (events 7, 13 and 22). Filtering occurred in the Python consumer after reading the events, not at the Kafka broker.

Experiment 6 — Optional CSV export

`consumer_csv.py` read 30 previously published events from `campus-buildings` and saved them to `campus_events.csv` with columns `event_id`, `sensor_id`, `temperature_c`, `humidity_pct`, and `timestamp`. PowerShell `Import-Csv` confirmed **30 data rows** (plus a header row). No additional producer run was required.

3. Answers to the nine report questions

Q1. What problem does Apache Kafka solve in a data-streaming architecture? Kafka decouples event producers from consumers. Producers publish records to a durable,

partitioned log, while multiple consumer applications can process the events independently and at different rates. This supports continuous processing, replay of retained events and scaling through partitions and consumer groups.

Q2. Define producer, consumer, broker, topic, partition, offset, and consumer group.

- **Producer:** application that publishes records to Kafka topics.
- **Consumer:** application that reads records from Kafka topics.
- **Broker:** Kafka server that stores and serves topic partitions.
- **Topic:** named logical stream of records.
- **Partition:** ordered append-only segment of a topic.
- **Offset:** position of a record within a specific partition.
- **Consumer group:** consumers sharing a group ID and coordinating partition assignments to divide consumption work.

Q3. What is the difference between a Kafka topic and a partition? A topic is a named logical collection of records; a partition is one of its ordered logs. A topic may have several partitions to distribute storage and processing. Each partition has its own offset sequence.

Q4. Why does the consumer use `auto_offset_reset='earliest'`? When does this setting apply? It instructs the consumer to begin at the earliest retained offset when its group has no valid committed position for a partition (for example, a new group). It does **not** force every restart to begin at the start when valid committed offsets already exist.

Q5. What is the purpose of the consumer group ID? What happens if a new group ID is used? The group ID identifies consumers that coordinate their partition assignments and committed offsets. Consumers in the same group share partition work. A new group has independent offsets and can read previously retained messages from the earliest offset when configured to do so. This was observed with `group-A`, `group-B`, and `group-shared`.

Q6. Why is JSON suitable for this exercise? Give one limitation. JSON is human-readable, easy to serialize and deserialize in Python, and supports named fields such as temperature, humidity and building. One limitation is additional text/serialization overhead compared with compact binary formats; without schema validation, field types and structure may also vary unexpectedly.

Q7. Explain how Docker Compose simplifies deployment compared with starting services manually. Docker Compose records the Kafka image, environment variables, port mapping, persistent volume and restart policy in `compose.yaml`. A single `docker compose up -d` command creates and starts the configured service reproducibly, reducing manual configuration steps.

Q8. What are the limitations of this single-broker educational setup compared with a production Kafka cluster? A single broker is a single point of failure; with replication factor 1, partitions have no replicas on other brokers. There is no cross-broker load distribution or failover. The setup also lacks production-grade security, monitoring, capacity planning and demonstrated high-load performance. Three partitions improve potential consumer parallelism but do not eliminate the single-broker risk.

Q9. Attach evidence showing the running container, topic description, producer output, and consumer output. The following screenshots document the successful execution of the practical work.

Figure 1 (`docker-compose-status.png`): The `tp05-kafka` container is running, and Kafka topics are listed.

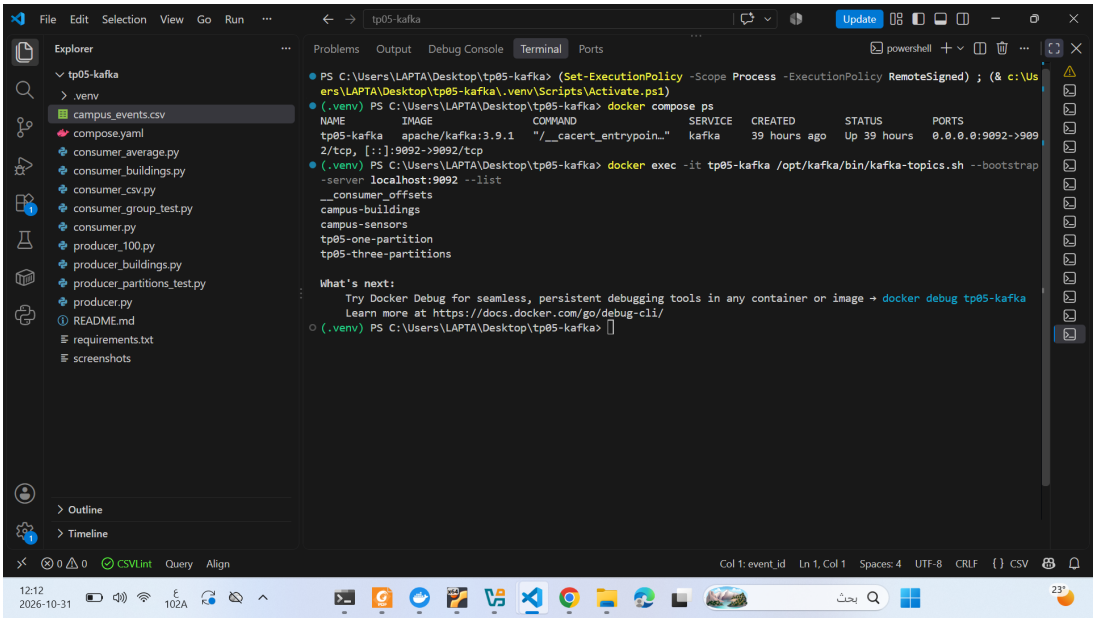


Figure 1: Docker Compose status

Figure 2 (`topic-description.png`): The `campus-sensors` topic has three partitions and replication factor 1.

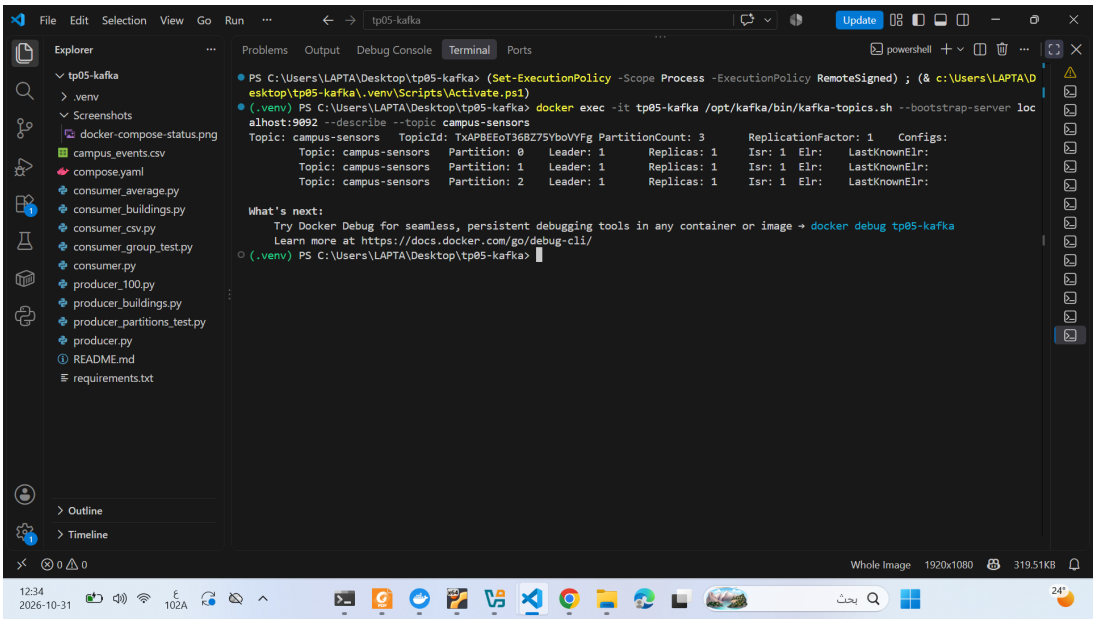


Figure 2: Kafka topic description

Figure 3 (`producer-output.png`): The Python producer publishes 30 JSON sensor events to Kafka.

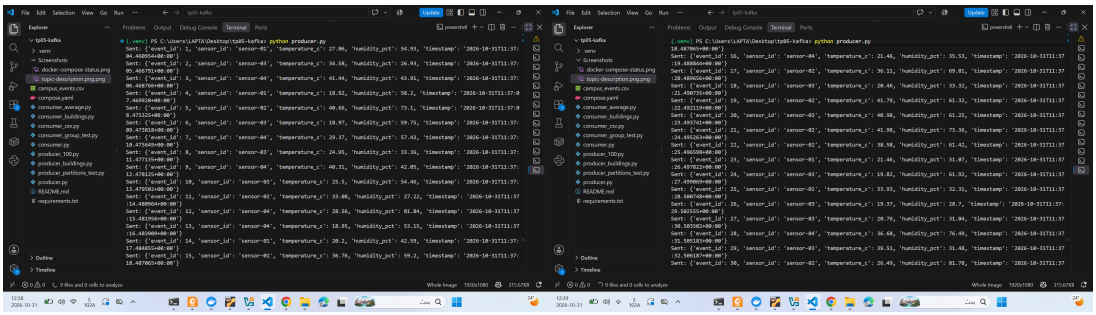


Figure 3: Producer output

Figure 4 (`consumer-output.png`): The Python consumer receives sensor readings and reports high-temperature alerts.

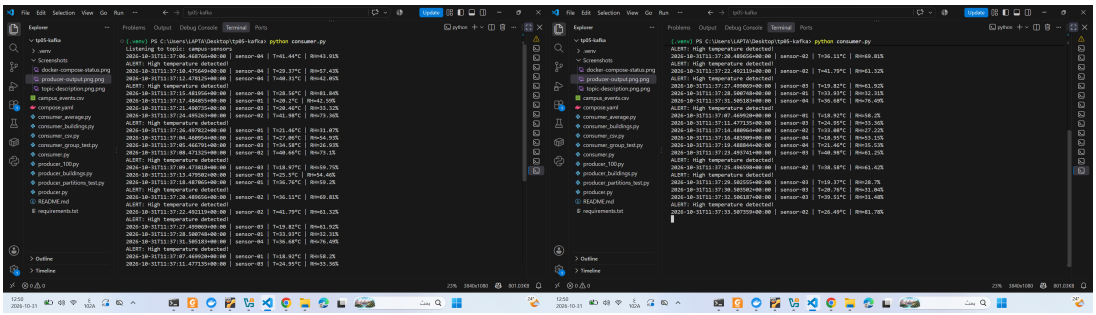


Figure 4: Consumer output

4. Problems encountered and solutions

Issue	Explanation and resolution
`localhost:9092` showed `ERR_EMPTY_RESPONSE` in Chrome	Port 9092 speaks the Kafka protocol, not HTTP. Verified broker operation using `docker compose ps` and Kafka CLI commands instead of a browser.
Cannot change a 3-partition topic down to 1 partition	Kafka does not support decreasing partition count directly. Created separate one- and three-partition experimental topics without changing the original topic.
Shared-group consumers initially showed no new messages	Previously committed offsets can leave no unread messages. Verified the two active members and their partition assignments using Kafka consumer-group CLI.
`KeyboardInterrupt` after Ctrl+C	Normal interruption of a continuously listening consumer, not a Kafka failure.
Event timestamps appeared out of order in consumer output	Records were read from different partitions. Kafka guarantees ordering per partition, not across all partitions.

5. Conclusion

The practical work successfully implemented a Kafka-based smart-campus event-streaming prototype. The experiments demonstrated publishing and consuming JSON events, threshold alerts, cumulative averages, consumer-group behavior, partition distribution, application-level filtering and CSV export. The single-broker deployment was sufficient for learning Kafka fundamentals, but does not provide production-level availability or fault tolerance.